Agent Swarm - Deployment en Infraestructura Propia

Fecha: 2026-03-21 **Autor:** Cesar / Claude Code **Estado:** Plan de infraestructura

---

1. De Scripts a Servicio

Hoy (scripts en covacha-projects)

```
covacha-projects/scripts/agent_memory/
├── 15 archivos .py (lógica)
├── 254 tests
└── Se ejecutan manualmente: python covacha_cli.py ...
```

Problemas:

- Hay que estar en el directorio correcto
- Cada máquina necesita clonar covacha-projects
- No hay API REST (solo CLI)
- No hay dashboard web
- No hay CI/CD dedicado
- No es instalable (`pip install` o `npm install`)

Target (servicio en AWS)

```
┌──────────────────────────────────────────────────────────────────┐
│                        AWS (tu cuenta)                           │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐  │
│  │ DynamoDB     │  │ Lambda       │  │ S3 + CloudFront      │  │
│  │ covacha-     │  │ dispatcher   │  │ dashboard.           │  │
│  │ agent-memory │  │ sync-github  │  │ swarm.superpago.     │  │
│  │ (ya existe)  │  │ stale-check  │  │ com.mx               │  │
│  └──────────────┘  └──────────────┘  └──────────────────────┘  │
│          │                  │                                   │
│          │     ┌────────────────────────────┐                  │
│          │     │  API Gateway + WAF         │                  │
│          │     │  swarm-api.superpago.com.mx │                  │
│          │     └────────────────────────────┘                  │
│          │                  │                                   │
└──────────┼──────────────────┼──────────────────────────────────┘
           │                  │
     ┌─────▼──────────────────▼─────┐
     │      Nodos (máquinas dev)    │
     │                              │
     │  ┌──────────┐  ┌──────────┐  │
     │  │ covacha  │  │ covacha  │  │
     │  │ CLI      │  │ MCP      │  │
```

```
│  │ (Go/Rust) │ │ Server    │  │
│  └──────────┘ └──────────┘   │
│                              │
│  Instalado via:              │
│  brew install covacha        │
│  o npm install -g covacha    │
└──────────────────────────────┘
```

---

2. Repos Nuevos

2.1 `baatdigital/covacha-swarm` — Backend (API + Lambdas)

```
covacha-swarm/
├── src/
│   ├── api/                      # API REST (Flask o FastAPI)
│   │   ├── routes/
│   │   │   ├── tenants.py        # CRUD tenants
│   │   │   ├── workspaces.py     # CRUD workspaces
│   │   │   ├── nodes.py          # Register, heartbeat, discover
│   │   │   ├── tasks.py          # Claim, release, find_next
│   │   │   ├── messages.py       # Send, read, mark_as_read
│   │   │   ├── contracts.py      # Publish, search
│   │   │   ├── dependencies.py   # Add, check, resolve
│   │   │   ├── evaluations.py    # Evaluate ticket
│   │   │   └── dashboard.py      # Stats, swarm summary
│   │   ├── middleware/
│   │   │   ├── auth.py           # API key + tenant isolation
│   │   │   └── rate_limit.py     # Rate limiting por tenant
│   │   └── app.py                # Flask/FastAPI app
│   │
│   ├── core/                     # Lógica de negocio (migrada de scrip
│   │   ├── tenant_manager.py
│   │   ├── node_registry.py
│   │   ├── dependency_manager.py
│   │   ├── message_bus.py
│   │   ├── contract_registry.py
│   │   ├── ticket_evaluator.py
│   │   ├── role_manager.py
│   │   ├── task_finder.py
│   │   └── prompt_generator.py
│   │
│   ├── lambdas/                  # Funciones Lambda
│   │   ├── dispatcher.py         # Asigna tareas a nodos idle (cada 5
│   │   ├── sync_github.py        # Sync GitHub Board → DynamoDB (cada
│   │   ├── stale_checker.py      # Detecta nodos stale, libera locks (
│   │   └── metrics.py            # Calcula métricas del swarm (cada 15
│   │
│   └── mcp/                      # MCP Server (TypeScript)
│       ├── server.ts             # MCP server setup
│       ├── tools/                # 14 tools wrapping la API REST
│       └── package.json
```

```
│
├── infra/                          # CloudFormation / CDK
│   ├── dynamodb.py                 # Tabla + GSIs
│   ├── api_gateway.py              # API Gateway + WAF
│   ├── lambdas.py                  # Lambda functions
│   └── dashboard.py                # S3 + CloudFront
│
├── tests/
│   ├── unit/                       # Migrados de scripts/agent_memory/te
│   ├── integration/                # Tests contra DynamoDB local
│   └── e2e/                        # Tests contra API desplegada
│
├── .github/workflows/
│   ├── ci.yml                      # Tests + lint
│   ├── deploy-api.yml              # Deploy API a EC2
│   ├── deploy-lambdas.yml          # Deploy Lambdas
│   └── deploy-dashboard.yml        # Deploy dashboard a S3
│
├── Procfile                        # Para EC2 (gunicorn)
├── requirements.txt
├── pyproject.toml
└── README.md
```

2.2 `baatdigital/covacha-cli` — CLI instalable (Go o Rust)

```
covacha-cli/
├── cmd/
│   └── covacha/
│       └── main.go                 # Entry point
├── internal/
│   ├── api/                        # HTTP client para covacha-swarm API
│   │   └── client.go
│   ├── config/                     # ~/.covacha-node.yml
│   │   └── node_config.go
│   ├── commands/                   # Subcommands
│   │   ├── init.go
│   │   ├── start.go                # Agent Loop
│   │   ├── stop.go
│   │   ├── status.go
│   │   ├── claim.go
│   │   ├── release.go
│   │   ├── msg.go
│   │   ├── deps.go
│   │   ├── contract.go
│   │   ├── eval.go
│   │   ├── tenant.go
│   │   ├── workspace.go
│   │   └── migrate.go
│   └── loop/                       # Agent Loop daemon
│       └── loop.go
├── .goreleaser.yaml                # Cross-compile + homebrew tap
├── go.mod
└── README.md
```

Distribución:

```
# Homebrew (macOS + Linux)
brew install baatdigital/tap/covacha

# Binary directo
curl -fsSL https://github.com/baatdigital/covacha-cli/releases/latest/d

# npm wrapper (para quien prefiera Node)
npm install -g @baatdigital/covacha
```

2.3 `baatdigital/covacha-mcp` — MCP Server (TypeScript)

```
covacha-mcp/
├── src/
│   ├── index.ts                  # Entry point
│   ├── server.ts                 # MCP server con @modelcontextprotoco
│   ├── tools/
│   │   ├── task-tools.ts         # claim, release, bootstrap, find_nex
│   │   ├── message-tools.ts      # send, read
│   │   ├── contract-tools.ts     # publish, get, search
│   │   ├── dep-tools.ts          # add, check, resolve
│   │   ├── eval-tools.ts         # evaluate ticket
│   │   ├── role-tools.ts         # role actions
│   │   ├── workspace-tools.ts    # list, switch, create
│   │   └── swarm-tools.ts        # status, heartbeat
│   ├── api-client.ts             # HTTP client para covacha-swarm API
│   └── config.ts                 # Lee ~/.covacha-node.yml
├── package.json
├── tsconfig.json
└── README.md
```

Instalación:

```
# Como plugin de Claude Code
claude plugin marketplace add baatdigital/covacha-mcp
claude plugin install covacha-mcp

# Como MCP server genérico
npm install -g @baatdigital/covacha-mcp
claude mcp add covacha -- covacha-mcp
```

---

3. API REST (covacha-swarm)

3.1 Endpoints

```
Base: https://swarm-api.superpago.com.mx/v1

# Auth: API key en header X-API-Key
# Tenant isolation: tenant_id del API key, no se pasa como parámetro

# ─── Nodes ───
```

```
POST   /nodes/register            # Registrar nodo
POST   /nodes/heartbeat           # Ping de vida
GET    /nodes                     # Listar nodos del tenant
GET    /nodes/:node_id            # Info de un nodo
DELETE /nodes/:node_id            # Deregistrar nodo

# ─── Tasks ───
POST   /workspaces/:ws/tasks/:id/claim      # Reclamar tarea
POST   /workspaces/:ws/tasks/:id/release     # Liberar tarea
GET    /workspaces/:ws/tasks/next            # Siguiente tarea para est
GET    /workspaces/:ws/tasks                 # Listar tareas

# ─── Messages ───
POST   /workspaces/:ws/messages              # Enviar mensaje
GET    /workspaces/:ws/messages/unread       # Mensajes no leídos
POST   /workspaces/:ws/messages/:id/read     # Marcar como leído

# ─── Dependencies ───
POST   /workspaces/:ws/deps                  # Agregar dependencia
GET    /workspaces/:ws/deps/:task_id         # Ver dependencias de una
GET    /workspaces/:ws/deps/:task_id/status  # ¿Está desbloqueada?

# ─── Contracts ───
POST   /workspaces/:ws/contracts             # Publicar contrato
GET    /workspaces/:ws/contracts             # Buscar contratos
GET    /workspaces/:ws/contracts/:id         # Ver contrato

# ─── Evaluations ───
POST   /workspaces/:ws/tasks/:id/evaluate    # Evaluar ticket

# ─── Workspaces ───
POST   /workspaces                           # Crear workspace
GET    /workspaces                           # Listar workspaces
POST   /workspaces/discover                  # Auto-discover desde GitH
DELETE /workspaces/:ws                       # Eliminar workspace
POST   /workspaces/:ws/sync                  # Forzar sync con GitHub

# ─── Tenant Admin ───
GET    /tenant                               # Info del tenant
PATCH  /tenant                               # Actualizar config
GET    /tenant/stats                         # Métricas del tenant

# ─── Dashboard ───
GET    /dashboard/summary                    # Resumen para el dashboar
GET    /dashboard/timeline                   # Timeline de eventos
```

3.2 Autenticación

```
Cada tenant tiene 1+ API keys.
La API key determina el tenant (no se pasa como parámetro).

Header: X-API-Key: cvh_live_a8f3b2c1...
```

```
DynamoDB:
  PK: APIKEY#cvh_live_a8f3b2c1
  SK: META
  tenant_id: baatdigital
  scope: ["admin"]  o ["node"]  o ["readonly"]
  created_by: cesar@baatdigital.com
  created_at: ...

Scopes:
  admin   — Todo (CRUD tenants, workspaces, keys)
  node    — Operaciones de nodo (claim, release, heartbeat, messages)
  readonly — Solo lectura (dashboard, status)
```

---

4. Lambdas (jobs automáticos)

4.1 dispatcher (cada 5 min)

```
# Evalúa el board y sugiere asignaciones
def handler(event, context):
    tenants = list_active_tenants()
    for tenant in tenants:
        workspaces = list_workspaces(tenant["tenant_id"])
        for ws in workspaces:
            idle_nodes = get_idle_nodes(tenant["tenant_id"])
            available_tasks = get_unblocked_tasks(tenant["tenant_id"],

            for node in idle_nodes:
                best_task = find_best_task_for_node(available_tasks, no
                if best_task:
                    send_message(
                        tenant_id=tenant["tenant_id"],
                        workspace_id=ws["workspace_id"],
                        from_node="dispatcher",
                        msg_type="task_suggested",
                        payload={"task_id": best_task["number"], "score
                    )
```

4.2 sync_github (cada 5 min)

```
# Sincroniza GitHub Projects → DynamoDB
def handler(event, context):
    tenants = list_active_tenants()
    for tenant in tenants:
        sync_interval = get_plan_sync_interval(tenant["plan"])
        if time_since_last_sync(tenant) < sync_interval:
            continue
        sync_all_workspaces(tenant["tenant_id"])
```

4.3 stale_checker (cada 2 min)

```
# Detecta nodos sin heartbeat y libera sus locks
```

```
def handler(event, context):
    tenants = list_active_tenants()
    for tenant in tenants:
        stale = check_stale_nodes(tenant["tenant_id"])
        for node in stale:
            release_node_task(node)
            send_message("node_stale", node["node_id"])
```

4.4 metrics (cada 15 min)

```
# Calcula métricas: tasks/día, tiempo promedio, throughput
def handler(event, context):
    tenants = list_active_tenants()
    for tenant in tenants:
        metrics = calculate_metrics(tenant["tenant_id"])
        store_metrics(tenant["tenant_id"], metrics)
```

---

5. Dashboard Web

5.1 Stack

```
Frontend: HTML + Alpine.js + Tailwind CSS (SPA estática)
Backend: API REST (covacha-swarm)
Hosting: S3 + CloudFront
URL: dashboard.swarm.superpago.com.mx
```

5.2 Vistas

```
┌──────────────────────────────────────────────────────────────────────┐
│  Covacha Swarm Dashboard          [baatdigital] [superpago]          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐            │
│   │ 3 Nodes  │  │ 12 Tasks │  │ 4 Done   │  │ 2 Blocked│            │
│   │  Active  │  │  Todo    │  │  Today   │  │          │            │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘            │
│                                                                      │
│   ┌──────────────────────────────────────────────────────┐           │
│   │  Nodes                                                │           │
│   │  ┌──────────────────┬────────┬──────┬──────────────┐ │           │
│   │  │ cesar-macbook    │ tech_  │ BUSY │ ISS-043      │ │           │
│   │  │                  │ lead   │      │ 2m ago       │ │           │
│   │  ├──────────────────┼────────┼──────┼──────────────┤ │           │
│   │  │ casp-mac-studio  │ dev    │ IDLE │ —            │ │           │
│   │  │                  │        │      │ 30s ago      │ │           │
│   │  ├──────────────────┼────────┼──────┼──────────────┤ │           │
│   │  │ casp-mac-mini    │ tester │ BUSY │ ISS-041      │ │           │
│   │  │                  │        │      │ 5m ago       │ │           │
│   │  └──────────────────┴────────┴──────┴──────────────┘ │           │
│   └──────────────────────────────────────────────────────┘           │
│                                                                      │
```

```
│   ┌──────────────────────────────────────────────────────┐       │
│   │  Recent Messages                                      │       │
│   │  cesar-macbook: task_completed ISS-043   2m ago       │       │
│   │  casp-mac-mini: qa_approved ISS-041      5m ago       │       │
│   │  dispatcher: task_suggested ISS-048      5m ago       │       │
│   └──────────────────────────────────────────────────────┘       │
│                                                                   │
│   ┌──────────────────────────────────────────────────────┐       │
│   │  Task Pipeline                                        │       │
│   │  TODO (8) → IN PROGRESS (3) → QA (1) → DONE (12)      │       │
│   └──────────────────────────────────────────────────────┘       │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

---

6. Infraestructura AWS (CloudFormation/CDK)

6.1 Recursos

```
Resources:

  # DynamoDB (ya existe, solo agregar GSIs si faltan)
  AgentMemoryTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: covacha-agent-memory
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: PK
          AttributeType: S
        - AttributeName: SK
          AttributeType: S
      KeySchema:
        - AttributeName: PK
          KeyType: HASH
        - AttributeName: SK
          KeyType: RANGE
      TimeToLiveSpecification:
        AttributeName: ttl
        Enabled: true

  # API Gateway
  SwarmAPI:
    Type: AWS::ApiGatewayV2::Api
    Properties:
      Name: covacha-swarm-api
      ProtocolType: HTTP
      CorsConfiguration:
        AllowOrigins: ["https://dashboard.swarm.superpago.com.mx"]
        AllowMethods: ["GET", "POST", "PATCH", "DELETE"]

  # Lambda: Dispatcher
```

```yaml
DispatcherFunction:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: covacha-swarm-dispatcher
    Runtime: python3.11
    Handler: dispatcher.handler
    Timeout: 60
    MemorySize: 256
    Environment:
      Variables:
        DYNAMO_TABLE: covacha-agent-memory

# EventBridge Rule: cada 5 min
DispatcherSchedule:
  Type: AWS::Events::Rule
  Properties:
    ScheduleExpression: rate(5 minutes)
    Targets:
      - Id: dispatcher
        Arn: !GetAtt DispatcherFunction.Arn

# Lambda: Stale Checker
StaleCheckerFunction:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: covacha-swarm-stale-checker
    Runtime: python3.11
    Handler: stale_checker.handler
    Timeout: 30

StaleCheckerSchedule:
  Type: AWS::Events::Rule
  Properties:
    ScheduleExpression: rate(2 minutes)
    Targets:
      - Id: stale-checker
        Arn: !GetAtt StaleCheckerFunction.Arn

# Lambda: GitHub Sync
SyncGithubFunction:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: covacha-swarm-sync-github
    Runtime: python3.11
    Handler: sync_github.handler
    Timeout: 120

# S3: Dashboard
DashboardBucket:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: covacha-swarm-dashboard
```

```
  # CloudFront: Dashboard
  DashboardDistribution:
    Type: AWS::CloudFront::Distribution
    Properties:
      DistributionConfig:
        Origins:
          - DomainName: !GetAtt DashboardBucket.DomainName
            Id: dashboard-origin
            S3OriginConfig:
              OriginAccessIdentity: ""
        DefaultCacheBehavior:
          TargetOriginId: dashboard-origin
          ViewerProtocolPolicy: redirect-to-https
        Aliases:
          - dashboard.swarm.superpago.com.mx
```

6.2 URLs

Servicio	URL	Tipo
API	swarm-api.superpago.com.mx	API Gateway
Dashboard	dashboard.swarm.superpago.com.mx	CloudFront + S3
DynamoDB	covacha-agent-memory (us-east-1)	Ya existe

---

7. CI/CD

7.1 covacha-swarm (API + Lambdas)

```
# .github/workflows/deploy.yml
name: Deploy Swarm API
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/ -v --cov --cov-fail-under=98

  deploy-api:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to EC2 via CodeDeploy
        run: |
          aws deploy create-deployment \
            --application-name covacha-swarm \
```

```
            --deployment-group-name production \
            --github-location repository=baatdigital/covacha-swarm,comm

  deploy-lambdas:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Package and deploy lambdas
        run: |
          cd src/lambdas
          zip -r dispatcher.zip dispatcher.py ../core/
          aws lambda update-function-code --function-name covacha-swarm
          # Repeat for each lambda
```

7.2 covacha-cli (Go binary)

```
# .github/workflows/release.yml
name: Release CLI
on:
  push:
    tags: ['v*']

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: '1.22'
      - uses: goreleaser/goreleaser-action@v5
        with:
          args: release --clean
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          HOMEBREW_TAP_TOKEN: ${{ secrets.HOMEBREW_TAP_TOKEN }}
```

7.3 covacha-mcp (npm package)

```
# .github/workflows/publish.yml
name: Publish MCP Server
on:
  push:
    tags: ['v*']

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci && npm test && npm run build
```

```
      - run: npm publish --access public
        env:
          NODE_AUTH_TOKEN: ${{ secrets.NPM_TOKEN }}
```

---

8. Plan de Migración

Fase A: Crear repos (1 día)

```
gh repo create baatdigital/covacha-swarm --private
gh repo create baatdigital/covacha-cli --private
gh repo create baatdigital/covacha-mcp --private
gh repo create baatdigital/homebrew-tap --public  # Para brew install
```

Fase B: Migrar lógica a covacha-swarm (2-3 días)

1. Copiar `scripts/agent_memory/*.py` → `covacha-swarm/src/core/`
2. Crear API REST wrapping los módulos existentes
3. Copiar tests → `covacha-swarm/tests/unit/`
4. Agregar tests de integración (DynamoDB local)
5. Deploy a EC2 (misma VPC que los otros backends)

Fase C: CLI en Go (3-5 días)

1. Implementar CLI en Go (mismo UX que covacha_cli.py)
2. HTTP client que habla con la API de covacha-swarm
3. Agent Loop en Go (más eficiente que Python para daemon)
4. GoReleaser para cross-compile + homebrew tap
5. `brew install baatdigital/tap/covacha`

Fase D: MCP Server en TypeScript (2-3 días)

1. 14 tools con @modelcontextprotocol/sdk
2. Cada tool llama a la API REST de covacha-swarm
3. Lee `~/.covacha-node.yml` para config local
4. Publicar en npm + Claude Code plugin marketplace

Fase E: Lambdas + Dashboard (2-3 días)

1. Deploy lambdas (dispatcher, sync, stale checker, metrics)
2. EventBridge schedules
3. Dashboard SPA (Alpine.js + Tailwind)
4. Deploy a S3 + CloudFront

Fase F: Deprecar scripts/ (1 día)

1. Agregar warning en covacha_cli.py: "Migrado a covacha-swarm. Instala: brew install baatdigital/tap/covacha"
2. Documentar en CLAUDE.md la nueva ubicación
3. Mantener scripts/ 30 días para backward compat, luego eliminar

---

9. Costo Estimado AWS

Recurso	Uso estimado	Costo/mes
DynamoDB (PAY_PER_REQUEST)	~6 WCU, ~50 RCU	~$1-2
Lambda: dispatcher (5 min)	~8,640 invocaciones/mes	~$0.02
Lambda: stale_checker (2 min)	~21,600 invocaciones/mes	~$0.05
Lambda: sync_github (5 min)	~8,640 invocaciones/mes	~$0.02
Lambda: metrics (15 min)	~2,880 invocaciones/mes	~$0.01
API Gateway	~100K requests/mes	~$1
S3 (dashboard)	<1 GB	~$0.02
CloudFront (dashboard)	<10 GB transfer	~$1
EC2 (API, si no usas Lambda)	t3.micro shared	~$8 (o $0 si comparte EC2 existente)
Total		**~$2-10/mes**

---

10. Resultado Final

```
# Un dev nuevo en cualquier empresa:
brew install baatdigital/tap/covacha
covacha init
covacha start

# Un admin configura el tenant:
covacha tenant create --name mi-empresa --github-org mi-org
covacha workspace discover --tenant mi-empresa

# Dashboard:
open https://dashboard.swarm.superpago.com.mx

# Claude Code con MCP:
claude plugin marketplace add baatdigital/covacha-mcp
claude plugin install covacha-mcp
# → 14 tools disponibles automáticamente
```

Todo en tu infraestructura. Zero dependencia externa excepto GitHub (que ya usas).